

Paradigmas de programación

Informe N°1

Alvaro Benjamín Reyes Encalada

Profesor:
Roberto Gonzalez

Santiago - Chile
2021

Tabla de Contenido

Capítulo 1. Introducción.....	3
Capítulo 2. Marco teórico	4
2.1 Paradigmas.	4
2.2 Programación funcional.	4
2.3 Características de la programación funcional	5
2.4 Scheme	5
2.5 Problema	5
2.6 Características del problema.....	6
2.7 Diseño de la solución.....	6
Capítulo 3. Desarrollo de la aplicación.....	7
3.1 Generalidades previas.....	7
3.2 TDA	7
3.3 Funciones	8
Capítulo 4. Análisis de resultados	9
Capitulo 5. Instrucciones de uso	10
Capítulo 6. Conclusiones	11
Capítulo 7. Referencias	12

Capítulo 1. Introducción

A los alumnos de ingeniería en informática de la Universidad de Santiago de Chile, dentro del transcurso del curso llamado Paradigmas de la Programación, se les ha propuesto dar solución a un mismo problema visto de distintos puntos de vista, es decir, cambiar la forma en la cual se le da solución al problema, pero más que en buscar aquellas formas como un posible cambio, un paradigma es una forma de pensar, por lo que la dificultad de este curso, no es tanto solucionar el problema, sino, en cómo cambiar la forma en la cual hay que pensar para dar solución al mismo.

Dentro del siguiente informe se procederá a explicar cómo se ha dado una solución el problema expuesto, pero con la diferencia, de que esta solución ahora se ha propuesto desde un modelo de pensamiento distinto, un paradigma funcional, y como su nombre lo indica, se basa paradigma de programación declarativa basado en el uso de verdaderas funciones matemáticas. En este estilo de programación las funciones son ciudadanas de primera clase, porque sus expresiones pueden ser asignadas a variables como se haría con cualquier otro valor; además de que pueden crearse funciones de orden superior.

Capítulo 2. Marco teórico

2.1 Paradigmas.

La programación funcional tiene sus raíces en el cálculo lambda, un sistema formal desarrollado en los años 1930 para investigar la naturaleza de las funciones, la naturaleza de la computación y su relación con la recursión. Los lenguajes funcionales priorizan el uso de recursividad y aplicación de funciones de orden superior para resolver problemas que en otros lenguajes se resolverían mediante estructuras de control (por ejemplo, ciclos). Muchos lenguajes de programación funcionales pueden ser vistos como elaboraciones del cálculo lambda.

Algunos lenguajes funcionales también buscan eliminar los efectos secundarios; en contraste con la programación imperativa, que enfatiza los cambios de estado mediante la mutación de variables. Esto significa que dos expresiones sintácticas idénticas (llamadas a rutinas, por ejemplo) pueden producir significados distintos si dependen de algo que ha mutado: carecen de transparencia referencial. En los lenguajes funcionales más estrictos, en contraste, el valor generado por una función depende exclusivamente de los argumentos alimentados a la función. Al eliminar los efectos secundarios se puede entender y predecir el comportamiento de un programa mucho más fácilmente. Esta es una de las principales motivaciones para utilizar la programación funcional. Lo anterior también puede ser aprovechado para diseñar estrategias de evaluación que eviten repetir el cómputo de expresiones antes vistas, ahorrando tiempo de ejecución.

2.2 Programación funcional.

La programación funcional es conseguir lenguajes expresivos y matemáticamente elegantes, en los que no sea necesario bajar al nivel de la máquina para describir el proceso llevado a cabo por el programa, y evitar el concepto de estado del cómputo.

La secuencia de computaciones llevadas a cabo por el programa se rige única y exclusivamente por la reescritura de definiciones más amplias a otras cada vez más concretas y definidas. Si lo pudiésemos con un concepto conocido, este sería con una “estructura” dentro del lenguaje C, solo que esta “estructura” además de poseer atributos, posee métodos, que trabajan en base a estos atributos, por lo que lo hace una herramienta más robusta y versátil que la “estructura”.

2.3 Características de la programación funcional

Cada programa funcional tiene un tipo concreto de programación declarativa, la cual tiene las siguientes características:

- Definiciones de funciones matemáticas puras, sin estado interno ni efectos laterales.
- Valores inmutables.
- Uso profuso de la recursión en la definición de las funciones.
- Uso de listas como estructuras de datos fundamentales.
- Funciones como tipos de datos primitivos: expresiones lambda y funciones de orden superior.

2.4 Scheme

Según Domingo Gallardo, “Scheme es un dialecto de Lisp, es un lenguaje interpretado, muy expresivo y soporta varios paradigmas. Influenciado por el cálculo lambda. El desarrollo de Scheme ha sido lento, ya que la gente que estandarizó Scheme es muy conservadora en cuanto a añadirle nuevas características, porque la calidad ha sido siempre más importante que la utilidad empresarial. Por eso Scheme es considerado como uno de los lenguajes mejor diseñados de propósito general”. (Gallardo, Lenguajes y Paradigmas de Programación, curso 2020-21).

La orientación de este laboratorio se realizará bajo el lenguaje de programación llamado “DR.Racket” en su versión 8.2.

2.5 Problema

El proyecto a presentar, se basa en realizar un editor de texto colaborativo, en donde, se muestra cómo realizar ciertas operaciones que pueden realizar esta plataforma. Estos editores son documentos colaborativos por cualquiera de los usuarios, da lugar a una nueva versión del documento. Todas las versiones de un documento conforman lo que se denomina “historial de versiones”. (Laboratorio, 2021.02 – Paradigmas). Entonces, cómo realizar este editor de texto colaborativo en base a las operaciones a realizar (ver punto 2.6) es la problemática que tenemos para poder prestar una solución asociada a una red social.

2.6 Características del problema

Dentro de las operaciones que típicamente se pueden realizar en estas plataformas tecnológicas de redes sociales se encuentran:

1. Crear una cuenta
2. Iniciar sesión
3. Crear Documentos
4. Eliminar Documentos
5. Compartir Documentos
6. Asignar permisos sobre un documento
7. Editar un documento
 - ✓ Escribir
 - ✓ Cambiar estilos a los textos (ej: cursiva, negrita, etc.)
 - ✓ Buscar y reemplazar
 - ✓ Copiar texto
 - ✓ Pegar texto
 - ✓ Cortar texto
 - ✓ Crear títulos, subtítulos
 - ✓ Crear un índice

2.7 Diseño de la solución

Para resolver este problema se propone como solución desarrollar una red social y gestionar eventos principales que componen un editor de texto, la cual informe y muestre la cantidad de usuarios, creación de documentos, eliminación, compartir y una serie de características a diseñar para este laboratorio. Su diseño es acorde a las funciones por definir en un entorno de elementos bajo la operación descrita en el punto 2.6 (Operaciones).

Capítulo 3. Desarrollo de la aplicación

Dentro del siguiente capítulo se le otorgará una visión general del programa, como fue programado y que problemas resuelven sus partes.

3.1 Generalidades previas.

Para programar la aplicación se ha utilizado el lenguaje de programación denominado Scheme, el cual es un lenguaje funcional, lo que quiere decir que es posible incluir más de un paradigma dentro del lenguaje. Además, para la creación de vistas y main se incluirá para “project.rkt”. Este main cuenta con vistas que se encargan de interactuar con las funciones implementadas en los TDA.

3.2 TDA

Project.rkt tiene consigo el TDA principal la cual construye este editor. Principal porque se desarrolla el *ParadigmaDocs* bajo el entorno que corresponde la plataforma que alberga la sesión activa, usuarios, documentos, versiones de los mismos. la cual me permite hacer funciones en una estructura requerida por el laboratorio, que es la contención de sesión activa, para el traspaso de archivos o subida, por ejemplo.

En la Imagen 1, se muestra el constructor de la función principal ParadigmaDocs

```
; constructores
(define (paradigmadocs name date encryptFunction decryptFunction)
  (if (and (esName? name) (esDate? date) (esFuns? encryptFunction decryptFunction))
      (list name date encryptFunction decryptFunction '()) null))
```

Imagen 1. ParadigmaDocs

Una vez identificadas las funciones, las cuales serán representadas en la Imagen 2, toma como posesión los elementos que se ocuparan para poder implementar el editor, el cual define el login exitoso (user y pass), añadir archivos, poder compartir y eliminarlos y todo formado en una lista que se representará para los selectores desarrollados y obtener los elementos anterior mente dicho, todo esto bajo las funciones de pertenencia.

Los modificadores vienen a declarar la función que fue obtenido a través de los selectores, y poder así, ser accesible a cualquier función implementada.

3.3 Funciones

Para un TDA formado, definimos diferentes funciones que el Laboratorio N°1 de Paradigmas de Programación, 2021-02, nos entrega para su evaluación, las cuales son los siguientes:

1. Register
2. Login
3. Create
4. Share
5. Add
6. RevokeAllAccess
7. ParadigmaDocs -> string
8. EncryFn
9. EmptyGdoc

Capítulo 4. Análisis de resultados

Una vez terminado el proyecto, la funcionalidad cumple con los requisitos funcionales establecidos. A continuación, se procederá a evaluar cada uno de ellos con lo que la aplicación posee:

1. Una implementación de TDA.
2. Se han implementado funciones en base a los TDA (explicados en la unidad anterior).
3. Creación de `project.rkt`, para utilizar todas las funciones de “consulta” al TDA en conjunto.
4. Verificación de funciones entre sí, definidas por los TDA en cuestión, es decir y dar un ejemplo, para validar una creación o share en particular, si o si debe estar la función `register` implementada.
5. Realización de comentarios dentro del editor, como funciona cada elemento y parámetro de desarrollo en base a los prerequisites señalados.
6. Implementación de `encrypt` y `decrypt`, en función de encriptar y desencriptar todo el contenido o el texto.
7. “GDocs” es la versión posterior con más datos cada vez que añadimos datos al editor de texto para crear, eliminar o compartir.
8. Se implementaron parámetros “ParadigmaDocs -> String” y “EncryFn” como función dependiente a los obligatorios.
9. Función “`getActiveUser`”, se utiliza para obtener todos los usuarios activos a través del editor. La lista contiene detalles de muchos archivos, encuentra el usuario con el `username` y devuelve la lista de archivos. Función importante para “EmptyGDocs”.

Capítulo 5. Instrucciones de uso

Se solicita construir una red social en un lenguaje de programación llamado Scheme, almacenando operaciones que típicamente se pueden realizar en estas plataformas, las cuales se encuentran:

1. Register
2. Login
3. Create
4. Share
5. Add
6. RevokeAllAccess
7. ParadigmaDocs -> string
8. EncryFn
9. EmptyGdoc

El programa presentado permite encontrar cada función a consultar que indica propiamente tal la salida, las cuales las publicaciones, usuarios registrados, comparticiones dentro de una publicación.

El programa puede ser utilizado tanto en Linux como en Windows, con el mismo IDE.

Dr. Racket es el IDE para utilizar en su versión 8.2. la cual tenemos que ir a *file – Open* y cargamos nuestro archivo .rkt para realizar las consultas correspondientes.

Capítulo 6. Conclusiones

Para finalizar, nuevamente la mayor dificultad dentro del desarrollo de la aplicación fue la planificación del proyecto en base al nuevo paradigma, el cambio en la manera de pensar el desarrollo de una aplicación es tremendamente complicado, pero una vez que se ha podido establecer el nuevo paradigma el resto de las dificultades no es tan grande (dificultades como: aprender un lenguaje de programación nuevo, definición entre funciones).

Como se menciona en la sección de análisis de resultados, se ha podido entregar una aplicación, la cual funciona satisfactoriamente implementado el paradigma funcional, luego de terminar esta experiencia, se resaltan las facilidades otorgadas por el paradigma, en realidad, se resaltan las funciones a definir para la utilización de estas nuevas variables, ya que estos hacen mucho más accesible la resolución de problemas.

Otro aspecto a destacar es la planificación que conlleva la programación funcional, mediante los TDA, este paso alivia mucha carga de creación de funciones, es aquí donde se ve reflejada que una buena planificación puede alivianar mucho los problemas presentados a la hora de crear código.

Concluyendo, el paradigma funcional es una buena herramienta para crear soluciones a problemas, tanto por su planificación como por su función para diseñar un sin fin de plataformas, cómo juegos, redes sociales, aplicaciones.

Capítulo 7. Referencias

http://www.gedlc.ulpgc.es/docencia/lp/documentacion/GB_Scheme.pdf

<http://www.dccia.ua.es/dccia/inf/assignaturas/LPP/2010-2011/teoria/tema2.html>

<http://ceciliaurbina.blogspot.com/2010/11/scheme.html>

<https://www.inf.utfsm.cl/~noell/PLP-UCV/apunte04.pdf>

<https://www.inf.utfsm.cl/~mcloud/iwi-253/apuntes/apunte04-03-04-2x.pdf>

<https://www.youtube.com/watch?v=H3ExAU7QKt4> (todos los videos)

Apuntes USACH.